



Checklist de teste estático

Gabriel Castro Freitas ^{*1}, Luiz Gustavo França Barreto Silva ^{†1}, Ryan David de Oliveira Silva ^{‡1}, Orientador Pedro Arthur Melo ^{§1*}

¹ Engenharia de Software

Escola de Tecnologias

Universidade Católica do Salvador (UCSAL)

Av. Prof. Pinto de Aguiar, 2589 Pituaçu, CEP: 41740-090

Salvador/BA, Brasil

¹*{gabrielcastro.freitas@ucsal.edu.br, luizgustavofranca.silva, ryan.silva}@ucsal.edu.br*

^{1*}*{pedroam.nascimento}@pro.ucsal.edu.br*

Setembro 2026

*gabrielcastro.freitas@ucsal.edu.br

†luizgustavofranca.silva@ucsal.edu.br

‡ryan.silva@ucsal.edu.br

§pedroam.nascimento@pro.ucsal.br

1 Introdução

1.1 Considerações gerais

Este documento visa apresentar um diagnóstico detalhado acerca do cumprimento do código-fonte do projeto PokéSal com as normas de qualidade de software delimitadas em testes estáticos utilizando os parâmetros do Google.

Os desenvolvedores usaram ao longo do desenvolvimento o Eclipse e o IntelliJ como IDEs, onde extensões terceiras relacionadas ao checkstyle foram acopladas ao ambiente de desenvolvimento, mapeando o código-fonte em busca de erros que, embora não afetassem a aplicação em tempo de execução, violaram as normas de limpeza do código, sendo essencial inclusive para delegar adequadamente o trabalho para outro membro realizar a próxima contribuição.

O projeto compila conforme o Java 21 (SDK pré-requisitado para todo o código-fonte), sem erros de sintaxe que impossibilitem a aplicação de ser executada.

2 Análise dos componentes do código

Os desenvolvedores, esporadicamente auxiliados por usos pontuais de IA generativa, garantiram conformidade total à ferramenta de checkstyle que escaneou todas as dependências em Java do projeto, sob as regras da Google.

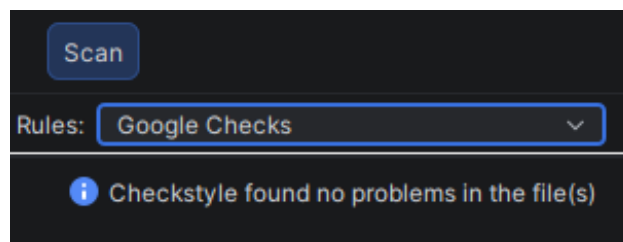


Figura 1 – Feedback do checkstyle tool integrado ao IntelliJ dentro do projeto.

2.1 Javadocs e comentários significativos

Uma explicação sucinta - sem ultrapassar 100 caracteres por linha - foi feita para todos os construtores e funções de feature dentro do código, referenciando claramente cada um dos parâmetros e eventuais retornos, quando presentes.

2.2 Tipagens e usos de constantes via enums

Todos os dados foram inicializados antes do uso, sendo sempre utilizadas após declaradas. Os desenvolvedores adotaram o uso de constantes semânticas expressas por enums para tornar a tipagem do projeto singular e personalizada.

2.2.1 Uso dos dados declarados

Atributos e variáveis sempre inicializados antes do uso e reatribuição de valores, contando com instâncias inicialmente vazias para atender logicamente aos requisitos da aplicação (como listas iniciadas com valor zero, permitindo a validação dos 3 golpes obrigatórios conforme o requisito).

```

public enum TipoElemental { 48 usages  Ryan David
    FOGO, 14 usages
    AGUA, 13 usages
    PLANTA; 13 usages

    /** Multiplicador aplicado quando ha vantagem elemental. */
    public static final double VANTAGEM_MULTIPLICADOR = 2.0; 3 usages

    /** Multiplicador aplicado quando ha desvantagem elemental. */
    public static final double DESVANTAGEM_MULTIPLICADOR = 0.5; 2 usages

    /** Multiplicador aplicado quando o combate elemental e neutro. */
    public static final double NEUTRO_MULTIPLICADOR = 1.0; 2 usages

```

Figura 2 – Classe de tipos elementais implementada com valores significativos para reuso em classes de entidades e serviços

2.2.2 Sobre o uso de downcasting e polimorfismo

Os princípios de orientação a objetos foram implementados sem downcastings inseguros ou inalcançáveis, por meio das entidades de terreno, efeitos e itens de batalha.

2.3 Acessibilidade de blocos de código

Ao longo dos processos de desenvolvimento e revisão contínua, todos os blocos de controle e variáveis primitivas não foram usadas com escopo inalcançável quando acionadas, permitindo o fluxo perfeito da lógica de negócio do projeto. Existem métodos com retornos finais vazios que funcionam como fallbacks alcançável, caso cenários de condições neutras fossem alcançados.

3 Aspectos problemáticos

Embora o esforço tenha sido contínuo para assegurar a qualidade do software a nível estático, alguns elementos ainda têm janela para melhorias.

3.1 Métodos sem utilização

Nas classes de entidades (pacote /model nas dependências do projeto), os getters e setters foram criados automaticamente englobando todos os atributos de cada objeto novo. No entanto, uma parcela desses metodos de aquisição de dados não foram utilizados na classe de serviço e nem na aplicação principal a nível de console.

Há também, dentro da classe de Batalha (presente nas dependências de serviços), dois métodos significativos (um inclusive relacionado à ação de ataque do PokéSal) e um construtor sem uso (sobrecarga de objetos desnecessária), uma vez que serve como uma instância da batalha com gerações aleatórias, uma lógica que não foi usada na versão executável final do programa.

3.2 Coerência nas abstrações

O código-fonte das entidades de modelo falha com o detalhe das abstrações: a classe generalista do tipo elemental do PokéSal é corretamente definida como abstrata, enquanto o golpe, entidade com a mesma aplicabilidade, foi mantida como classe concreta, embora apenas suas subclasses fossem efetivamente instanciadas via polimorfismo.